Universidade Federal Rural do Semi-Árido – UFERSA
Departamento de Computação – DC

Algoritmo RSA

Segurança Computacional
Prof. Paulo Henrique Lopes Silva

Roteiro

- Fundamentos.
- Geração de chaves.
- Encriptação e decriptação.

Fundamentos

- RSA é um algoritmo assimétrico desenvolvido por Ron Rivest, Adi Shamir e Leonard Adleman, pesquisadores do MIT.
  - É um sistema criptográfico muito usado atualmente (seguro).
- Exemplos de aplicações:
  - Assinatura digital de certificados e arquivos.
  - Comunicação entre entidades na Internet (chats, internet banking, comércio eletrônico, etc).
- Usa duas chaves (pública e privada) para encriptação e decriptação.

Fundamentos

- RSA se baseia na dificuldade de fatoração do produto entre dois números primos grandes.
  - Dados dois números primos grandes p e q, é fácil calcular n = p * q;
  - Mas é difícil calcular os fatores p e q a partir de n.
- RSA é um algoritmo (assim como outros) baseado em aritmética modular, que opera sobre corpos finitos.

Geração de chaves

- No RSA, as chaves são baseadas em números primos (quanto maior o número, mais segura é a chave).
- **Passo 1**: sortear dois números primos aleatórios p e q.
  - Forma usual: escolher dois números grandes e testar sua primalidade (teste probabilístico).
  - Por simplicidade, p = 11 e q = 13.
  - Números devem ser grandes.
    - Menor utilizado tinha 100 dígitos (RSA-100-330 bits).
    - Último quebrado foi o RSA-232 (768 bits).
    - Usados hoje: 617 dígitos (RSA-2048).
      - Chaves de 3072 e 4096 bits estão disponíveis.

Geração de chaves

- **Passo 2**: calcular módulo n.
  - n = p * q = 11 * 13 = 143.

Geração de chaves

- **Passo 3**: calcular a função totiente (ϕ ou phi) de Euler.
  - Calcular o número de coprimos à um número.
    - Dois números são coprimos (ou primos entre si) quando o MDC entre eles é igual a 1.
    - Por exemplo, o número totiente (ou seja, o número de coprimos) de 8 é 4. Por que?
      - MDC(8,1) = 1
      - MDC(8,2) = 2
      - MDC(8,3) = 1
      - MDC(8,4) = 4
      - MDC(8,5) = 1
      - MDC(8,6) = 2
      - MDC(8,7) = 1
  - Portanto existem 4 números que são coprimos ao 8: 1,3,5,7. Então o número totiente de 8 é 4.

Geração de chaves

- **Passo 3**: calcular a função totiente (ϕ ou phi) de Euler.
  - Calcular essa função totiente é bastante difícil.
  - Euler descobriu que quando se conhece os dois números primos que dão origem a um número, o totiente (número de coprimos) é:
    - phi(n) = (p – 1) * (q – 1).
  - No exemplo:
    - phi(143) = (11 – 1) * (13 – 1) = 10 * 12 = 120.

Geração de chaves

- **Passo 4**: calcular o expoente e (para a chave pública).
  - 1 < e < phi(n) e mdc(e, phi(n)) = 1.
  - Assim, e e phi(n) devem ser coprimos.
  - No exemplo, temos:
    - phi(n) = 120 = 10*12 = 2*5*2*6 = 2*5*2*2*3 = 2^3 * 3 * 5.
    - Por simplicidade, vamos escolher o 7 (menor primo que não divide 120).
    - Qualquer número que satisfaça o mdc(e, 120) = 1 poderia ser escolhido.

Geração de chaves

- **Passo 5**: calcular o expoente d (para a chave privada).
  - Tem-se que d*e ≡ 1 mod phi(n) (d é o inverso multiplicativo modular de e), com d < phi(n).
  - Ou seja, d*e mod phi(n) = 1.
  - Obtêm-se d pelo algoritmo de Euclides estendido.
  - No exemplo, usando a função abaixo, d = 103.

```
void buscarD(int e, int totiente){

     var i = 1;
     while (true){
             if (e * i % totiente == 1){
                 println("d é: "+i);
                 break;
             }
             i++;
     }
}
```

Geração de chaves

- **Passo 6**: as chaves pública e privada são formadas pelo módulo n e seus expoentes e e d.
  - K_{pub} = {e, n} = {7, 143}.
  - K_{pri} = {d, n} = {103, 143}.
  - Os outros valores importantes:
    - p = 11 e q = 13.
    - phi(n) = 120.

Cifragem

- Para a cifragem usa-se a chave pública (ou chave de codificação).
- $C = M^e \bmod n$.
- Exemplo: cifrar “ola”.
  - Pré-processamento: dividir a mensagem em blocos (“ola” = 111 108 97 em ASCII).
    - $111^7 \bmod 143 = 45$
    - $108^7 \bmod 143 = 4$
    - $97^7 \bmod 143 = 59$
    - C = 45 4 59

Decifragem

- Para a decifragem usa-se a chave privada (ou chave de decodificação).
- $M = C^d \bmod n$.
- Exemplo: C = 45 4 59.
  - $45^{103} \bmod 143 = 111$
  - $4^{103} \bmod 143 = 108$
  - $59^{103} \bmod 143 = 97$
  - M = 111 108 97

    ‘o’ ‘l’ ‘a’

RSA é seguro

- Números p e q na prática são grandes.
  - Manter os dois seguros.
- Cifragem e decifragem são executadas de maneira fácil pelo computador.
- Para descobrir a chave privada é preciso fatorar números muito grandes.
  - Computacionalmente inviável.

Observações práticas

- A segurança depende de n ser difícil de fatorar. Se um atacante fatorar n, ele recupera p, q, calcula phi(n) e então d.
  - Números p e q na prática são grandes.
  - Tamanhos: hoje recomenda-se no mínimo 2048 bits em n (muitos usam 3072 ou 4096 bits para maior segurança).
  - Gasto computacional elevado para gerar chaves.
- Chave privada pequena.
  - Deve-se garantir a escolha de um d suficientemente grande.
- Escolher primos.
  - Primos grandes resolvem, mas;
  - Se p e q são primos, mas |p – q| não, é fácil fatorar o n pelo algoritmo de Fermat.

Cuidados na implementação

- Lidar com números inteiros muito grandes.
  - Manter os dois seguros.
- Definir os blocos para cifragem.
  - Cada bloco deve ser menor que n (p * q).
  - Evitar blocos começando com zero.
- Na decifragem, saber quantos números representam cada letra.

Cuidados na implementação

- NUNCA use RSA puro (texto simples elevado a e) sem *padding*.
  - Use sempre padrões seguros: OAEP (para encriptação) e PSS (para assinatura).
- RSA é lento para grandes volumes:
  - Na prática usa-se RSA para trocar uma chave simétrica (ex.: AES) e depois usa-se AES para cifrar os dados.
- Proteja contra ataques de implementação (*timing attacks*, ataques por canal lateral) e gere primos aleatoriamente/seguramente.

Cuidados na implementação

- *Timing attacks* (ataques por tempo).
  - Exploração de variações no tempo que um sistema leva para executar operações (por exemplo, comparação de chaves, operações criptográficas) para inferir informação secreta.
    - Imagine que um servidor tem uma senha secreta, e quando alguém tenta fazer login, ele faz algo ingênuo, como:

```
boolean verificarSenha(String tentativa, String senhaCorreta) {
    for (int i = 0; i < senhaCorreta.length(); i++) {
        if (tentativa.charAt(i) != senhaCorreta.charAt(i)) {
            return false; // para imediatamente se encontrar diferença
        }
    }
    return true;
}
```

Cuidados na implementação

- *Timing attacks* (ataques por tempo).
  - Essa função compara caractere por caractere.
  - Assim que encontra uma diferença, ela sai do laço.
  - Isso significa que:
    - Se o primeiro caractere já está errado: a função retorna muito rápido.
    - Se o primeiro caractere está certo mas o segundo está errado: leva um pouquinho mais de tempo.
    - Se os dois primeiros estão certos, o tempo é ainda um pouco maior, e assim por diante.
    - O tempo total de execução varia de acordo com quantos caracteres iniciais estão certos.

Cuidados na implementação

- *Timing attacks* (ataques por tempo).
  - Um atacante pode:
    - Enviar muitas tentativas diferentes e medir o tempo de resposta de cada uma.
    - Descobrir qual tentativa demorou um pouquinho mais, o que indica que o servidor comparou mais caracteres antes de falhar.
  - A solução é fazer comparações de tempo constante:
    - Ou seja, comparar toda a string sempre até o final, mesmo que os primeiros caracteres já estejam errados, e só decidir no fim.

Cuidados na implementação

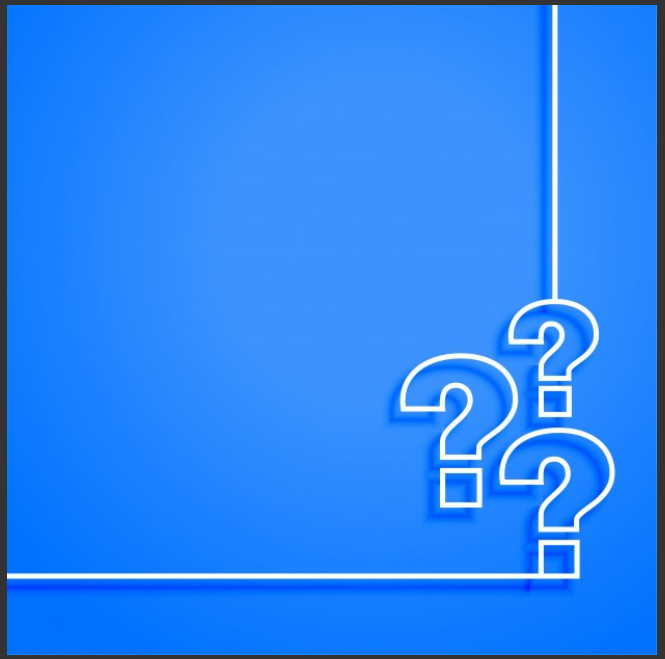
- *Side-channel attacks* (ataques por canal lateral).
  - Explora informações que são produzidas indiretamente durante a execução de um sistema, em vez de atacar diretamente o algoritmo criptográfico.
    - Tempo de execução;
    - Cache/CPU;
    - Consumo de energia;
    - Emissão eletromagnética;
    - Som;
    - Outros efeitos observáveis.

Cuidados na implementação

- *Side-channel attacks* (ataques por canal lateral).
  - O atacante observa alguma característica do comportamento do sistema e tenta utilizá-la para inferir informações secretas, como:
    - chaves criptográficas;
    - senhas;
    - dados processados;
    - informações sobre o estado interno de um algoritmo.
  - O ponto fundamental é:
    - O algoritmo pode estar matematicamente correto e ainda assim sua implementação vazar informações por meio de características observáveis durante sua execução.

Dúvidas?

- Acesse os canais de comunicação da disciplina.



Referências

- STALLINGS, William; BARRETO, Paulo Sérgio Licciardi Messeder; MISOCZKI, Rafael. **Criptografia e segurança de redes**: princípios e práticas. 6.ed. São Paulo: Pearson Education do Brasil, 2015.
- UNIVESP. Segurança da Informação - Aula 04 - Algoritmos assimétricos e certificação digital. 2018. (23m08s). Disponível em <https://www.youtube.com/watch?v=4xv0RD8T1qA>. Acesso em: 31 mar. 2021.
- MAZIERO, Carlos. Capítulo 3 – Criptografia assimétrica. 2019. Disponível em < http://wiki.inf.ufpr.br/maziero/lib/exe/fetch.php?media=sc:seg-texto-03.pdf>. Acesso em 10 abr. 2021.